Учебная программа «Робототехнический контроллер „Кибернетический“: моторы, сервоприводы, датчики» — расширенная

Объём: 68 часов (2 урока по 45 минут в неделю, 34 учебные недели).

Структура: I полугодие — контроллер, датчики, ESP32 (уроки 1–34); II полугодие — моторы, сервоприводы, манипулятор (уроки 35–68). Каждый модуль заканчивается практикумом или мини-проектом.

Цель курса

Научить учащихся программировать робототехнический контроллер «Кибернетический», подключать к нему датчики, моторы и сервоприводы и собирать из них работающие устройства: от мигающего светодиода до мобильного робота и манипулятора. По сравнению с программой на 34 часа здесь больше практикумов, есть гироскоп и магнитометр, Wi-Fi, одометрия, регуляторы, управление с телефона и основы кинематики.

Планируемые результаты

Учащиеся будут:

- знать устройство контроллера, назначение его узлов, правила питания и DIP-переключателя;
- писать программы в Arduino IDE для Arduino-части (ATmega328P) и ESP32;
- работать с цифровыми входами и выходами, шинами SPI, I2C, UART и шиной DXL (протокол Dynamixel 1.0);
- подключать и настраивать модули «Концевой выключатель», «Датчик линии», «Ультразвуковой дальномер», понимать причины ошибок измерения;
- управлять моторами с энкодерами, применять П- и PID-регуляторы, одометрию и гироскоп;
- работать с сервоприводами Feetech ST3215, собирать, калибровать и программировать манипулятор;
- решать прямую задачу кинематики для двухзвенного манипулятора;
- выполнять и защищать собственный проект.

Оборудование на рабочее место

- Робототехнический контроллер «Кибернетический», кабель USB-C, источник питания (аккумулятор с XT60 или блок питания с DC-разъёмом).
- Модули «Концевой выключатель», «Датчик линии», «Ультразвуковой дальномер», 3-пиновые кабели.
- Шасси с двумя моторами с энкодерами (II полугодие).
- Сервоприводы Feetech ST3215, манипулятор (на группу).
- Компьютер с Arduino IDE (с пакетом плат ESP32) и браузером Chrome / Edge / Яндекс Браузер.
- Смартфон (для уроков с Wi-Fi), поле с чёрной линией на белом фоне, рулетка, транспортир.

Памятка: DIP-переключатель SW12 (ON — слева)

№	ON = соединяет
1	ESP32 TX → Arduino RX
2	Arduino TX → ESP32 RX
3	ESP32 ↔ шина DXL

№	ON = соединяет
4	Arduino ↔ шина DXL
5	Включает светодиоды Arduino (выходы 74HC595)

Главные правила: **при прошивке Arduino DIP1 и DIP4 выключены; DIP3 и DIP4 одновременно не включать.**

Типовой ход урока (если не указано иное): повторение — 5 мин, новый материал — 15 мин, практика — 20 мин, итоги — 5 мин.

Примеры кода

Примеры к урокам лежат в папке `lessons` этого репозитория:

- `arduino_ide/Ardu8_NN_Название/` — скетчи для Arduino IDE (папку целиком открыть в Arduino IDE);
- `ar_bbl/Ardu8_NN_Название.xmlar` — те же задачи в блочной среде AR BBL (там, где это возможно);
- `Примеры_по_урокам.xlsx` и `README.md` — список примеров и соответствие урокам обеих программ.

В каждом уроке строка **«Примеры»** показывает, какие примеры к нему относятся. Номер `NN` у одного и того же примера одинаковый в обеих программах. Примеры намеренно простые и подробно прокомментированы; в начале каждого — плата, разъём USB и положение DIP-переключателя.

I полугодие — контроллер, датчики, ESP32 (34 ч)

Модуль 0. Введение (4 ч)

Урок 1. Знакомство с контроллером, техника безопасности

Цель: познакомиться с назначением контроллера и правилами безопасной работы.

Оборудование: контроллер (выключен), увеличенная карта разъёмов.

Ход урока: техника безопасности — 15 мин, что такое контроллер и где он применяется — 10 мин, осмотр платы — 15 мин, итоги — 5 мин.

Конспект:

- Контроллер — «мозг» робота: получает сигналы от датчиков, принимает решение по программе и управляет моторами.
- Правила: собирать и переключать провода только при выключенном питании; не класть плату на металлические предметы; не оставлять аккумулятор без присмотра на зарядке; при запахе гари или нагреве — сразу выключить питание.
- На плате два «мозга»: **Arduino-часть** (ATmega328P, как в Arduino Uno) и **ESP32** (с Wi-Fi). У каждого свой набор: 4 кнопки, 4 зелёных светодиода, RGB-светодиод. Верхняя группа — Arduino, нижняя (у модуля ESP32) — ESP32.
- Два контроллера моторов STM8 управляют четырьмя моторами с энкодерами. Также на плате есть датчик движения IMU, пищалка и интерфейс CAN.

Задание: найти в интернете три примера роботов и описать, какие у них датчики и исполнительные устройства.

Урок 2. Устройство платы

Цель: разобраться с питанием, разъёмами и DIP-переключателем.

Оборудование: контроллер, карта разъёмов, аккумулятор.

Ход урока: питание — 15 мин, разъёмы — 15 мин, DIP-переключатель — 10 мин, итоги — 5 мин.

Конспект:

- **Питание:** аккумулятор (XT60) или блок питания (DC) через красный выключатель → шина VCC (моторы, сервоприводы) → преобразователи 5 В и 3,3 В (логика).
- От USB питается только логика: можно программировать без аккумулятора, но моторы и сервоприводы работать не будут.
- **USB-C на левом краю:** верхний — Arduino, нижний — ESP32. **USB-C на верхнем краю** — выход питания с быстрой зарядкой для внешних устройств.
- Сверху — 3-пиновые разъёмы (GND, 5 В, сигнал) для A0–A2 и D2–D10; в центре — гребёнки как у Arduino Uno; справа — 4 разъёма моторов; снизу — разъёмы шины DXL (XP2, XP3), RJ12 (I2C и CAN), UART, I2C, SPI.
- DIP-переключатель соединяет узлы платы (см. памятку). Кнопка RESET перезапускает Arduino-часть и контроллеры моторов.

Задание: заполнить таблицу «разъём — назначение».

Урок 3. Первая программа

Цель: научиться загружать программу в Arduino-часть.

Оборудование: контроллер, USB-C, компьютер.

Настройка: DIP1 = OFF, DIP4 = OFF; плата *Arduino Uno*; верхний USB-C.

Примеры: Arduino IDE — `Ardu8_01_Blink`; AR BBL — `Ardu8_01_Blink`.

Конспект:

- Arduino IDE: выбрать плату и порт, «Проверить» компилирует программу, «Загрузить» записывает её в контроллер.
- Скетч: `setup()` выполняется один раз при запуске, `loop()` повторяется бесконечно.
- `pinMode(13, OUTPUT)`, `digitalWrite(13, HIGH/LOW)`, `delay(мс)` — Blink на светодиоде HL1 (D13).
- HL1 мигает и при обмене по SPI, так как D13 — тактовая линия SPI.
- Почему DIP1 и DIP4 выключены: у Arduino один `Serial`, общий для USB, шины моторов и ESP32. Если они подключены, загрузка не пройдёт.

Задание: сигнал SOS на HL1.

Урок 4. Переменные, циклы, монитор порта

Цель: освоить переменные, циклы и вывод данных.

Оборудование: контроллер, USB-C.

Примеры: Arduino IDE — `Ardu8_02_SerialCounter`, `Ardu8_03_LoopsPlotter`; AR BBL — `Ardu8_02_SerialCounter`.

Конспект:

- Переменные: `int` (целые числа), `float` (дробные), `bool` (истина/ложь). Объявляются до использования.
- Цикл `for (int i = 0; i < 10; i++)` повторяет действие заданное число раз; `if / else` — ветвление.
- `Serial.begin(115200)`, `Serial.print()` / `Serial.println()` — вывод в монитор порта. Скорость монитора должна совпадать со скоростью в программе.
- Плоттер рисует график, если в одной строке через пробел или запятую выводить числа (можно с подписями `имя:значение`).
- Светодиоды HL3 и HL4 мигают при приёме и передаче по `Serial`.

Задание: вывести в плоттер синусоиду `sin(i * 0.1)`.

Модуль 1. Встроенная периферия Arduino (12 ч)

Урок 5. Сдвиговый регистр 74HC595

Цель: понять, как регистр управляет восемью выходами по трём проводам.

Настройка: DIP5 = ON.

Примеры: Arduino IDE — `Ardu8_04_Leds595_Running`; AR BBL — `Ardu8_04_Leds595_Running`.

Конспект:

- Двоичное число: 8 бит = 1 байт, каждый бит — один выход.
- 74HC595 принимает байт по SPI (данные — D11, такт — D13), а импульс на защёлке (D8) выставляет его на выходы.
- Отправка: `digitalWrite(8, LOW); SPI.transfer(байт); digitalWrite(8, HIGH);` (в `setup()` — `SPI.begin()` и `pinMode(8, OUTPUT)`).
- Q0–Q3 — LED1–LED4, Q4 — синий, Q5 — зелёный, Q6 — красный цвет RGB, Q7 не используется.
- **0 — горит, 1 — погашен.** Всё погасить — `0xFF`, зажечь LED1 — `0xFE`.
- Без DIP5 = ON выходы регистра отключены. D8 и D13 заняты — для датчиков их не используем.

Задание: зажигать светодиоды по одному с интервалом 1 с.

Урок 6. Анимации на светодиодах

Цель: использовать битовые операции и массивы.

Примеры: Arduino IDE — `Ardu8_05_Leds595_Binary`, `Ardu8_06_Leds595_Animation`; AR BBL — `Ardu8_05_Leds595_Binary`.

Конспект:

- Сдвиг: `1 << n` — бит номер n. Инверсия: ~x. Объединение: `a | b`, проверка: `a & b`.
- Бегущий огонь: `for (n = 0; n < 4; n++) show(~(1 << n));`
- Двоичный счётчик: число от 0 до 15 прямо выводится на LED1–LED4.
- Массив кадров: `byte frames[] = {…};` — анимация как последовательность байтов.

Задание: придумать свою анимацию из 8 кадров.

Урок 7. Кнопки через 74HC165

Цель: читать кнопки через входной сдвиговый регистр.

Примеры: Arduino IDE — `Ardu8_07_Buttons165`; AR BBL — `Ardu8_07_Buttons165`.

Конспект:

- 74HC165 «фотографирует» состояние кнопок и передаёт его по SPI.
- Чтение: D10 → LOW (подключить регистр к MISO), импульс на D9 (LOW → HIGH), `SPI.transfer(0)`, D10 → HIGH.
- Кнопки BUT1–BUT4 (слева направо) — биты 0–3. **Нажатие даёт 1.**
- Вывод в двоичном виде: `Serial.println(b, BIN)`.
- Перенос на светодиоды с учётом инверсии: `leds = ~buttons`.

Задание: каждая кнопка зажигает свой светодиод.

Урок 8. События кнопок

Цель: различать нажатие, отпускание и долгое нажатие.

Примеры: Arduino IDE — `Ardu8_08_ButtonCounter`, `Ardu8_09_ButtonLongPress`; AR BBL — `Ardu8_08_ButtonCounter`.

Конспект:

- Состояние — «кнопка нажата сейчас». Событие — «кнопку только что нажали» (было 0, стало 1).
- Дребезг: при нажатии контакты замыкаются и размыкаются несколько миллисекунд. Решение — принимать новое состояние, только если оно держится 20–30 мс.
- Долгое нажатие: запомнить `millis()` в момент нажатия; если при отпускании прошло больше 1 с — это долгое нажатие.
- Удобно оформить чтение кнопок в функцию, которая возвращает событие.

Задание: короткое нажатие BUT1 увеличивает счётчик, долгое — сбрасывает.

Урок 9. RGB-светодиод

Цель: освоить смешение цветов.

Примеры: Arduino IDE — `Ardu8_10_RGB_Rainbow`; AR BBL — `Ardu8_10_RGB_Rainbow`.

Конспект:

- RGB — три светодиода в одном корпусе. Включая их в разных сочетаниях, получаем 7 цветов: красный + зелёный = жёлтый, зелёный + синий = голубой, красный + синий = пурпурный, все три = белый.
- Через 74HC595 каждый цвет можно только включить или выключить. Плавное изменение яркости получается программно: быстро включая и выключая цвет (программный ШИМ).
- Цвет удобно задавать константами: `RED = ~(1 << 6)` и т. д.

Задание: «радуга» — циклическая смена 7 цветов.

Урок 10. Конечный автомат

Цель: программировать устройство по состояниям.

Примеры: Arduino IDE — `Ardu8_11_TrafficLight`; AR BBL — `Ardu8_11_TrafficLight`.

Конспект:

- Конечный автомат: программа всегда в одном состоянии и переходит в другое по событию.

- Состояние хранится в переменной (`enum State {GREEN, YELLOW, RED}`), переходы описываются через `switch`.
- Ожидание без остановки программы: `if (millis() - t0 > 3000) { … }` — так кнопка работает и во время паузы.
- Перед написанием кода рисуем схему: кружки — состояния, стрелки — переходы с условиями.

Задание: светофор с кнопкой пешехода и миганием зелёного.

Урок 11. Практикум «Игра на реакцию»

Цель: закрепить кнопки, светодиоды и автомат.

Примеры: Arduino IDE — `Ardu8_12_ReactionGame`; AR BBL — `Ardu8_12_ReactionGame`.

Ход урока: постановка задачи — 5 мин, выполнение — 30 мин, турнир — 10 мин.

Конспект:

- Через случайное время (`random(1000, 5000)`) загорается RGB. Побеждает тот, кто первым нажмёт свою кнопку (BUT1 или BUT4).
- Нажатие до сигнала — фальстарт.
- Время реакции выводится в монитор порта, счёт — на светодиодах.
- Состояния: ОЖИДАНИЕ → СИГНАЛ → РЕЗУЛЬТАТ.

Задание: игра до трёх побед.

Урок 12. Шина I2C, акселерометр IMU

Цель: познакомиться с шиной I2C и акселерометром.

Примеры: Arduino IDE — `Ardu8_13_I2C_Scanner`, `Ardu8_14_IMU_Accel`.

Конспект:

- I2C — два провода: SDA (A4) и SCL (A5). На одной шине много устройств, у каждого свой адрес.
- Сканер I2C перебирает адреса и показывает, кто ответил. ICM-20948 отвечает по адресу **0x68**.
- Акселерометр измеряет ускорение по осям X, Y, Z. В покое он «чувствует» силу тяжести (1 g), поэтому по нему определяют наклон.
- Угол наклона: `atan2(ay, az) * 180 / PI`.
- Те же линии I2C выведены на XP5 и RJ12 XS9.

Задание: зажигать LED1 при наклоне влево и LED4 при наклоне вправо.

Урок 13. Гироскоп

Цель: измерить угол поворота платы.

Примеры: Arduino IDE — `Ardu8_15_IMU_Gyro`.

Конспект:

- Гироскоп измеряет угловую скорость (градусы в секунду), а не угол.
- Угол получается сложением: `угол += скорость * dt`, где dt — время между измерениями в секундах (интегрирование).
- В покое гироскоп показывает не ноль, а небольшое смещение. Перед началом работы его измеряют (среднее за 1–2 с неподвижности) и вычитают.

- Из-за ошибок угол постепенно «уплывает» (дрейф). Поэтому гироскоп хорош для коротких поворотов.

Задание: повернуть плату на 90° по транспортиру и сравнить с показаниями.

Урок 14. Магнитометр ⚠️

Цель: сделать электронный компас.

Примеры: Arduino IDE — `Ardu8_16_IMU_Compass`.

Конспект:

- Магнитометр AK09916 находится внутри ICM-20948 и имеет свой адрес **0x0C**. Он доступен после включения «обходного» режима (bypass) IMU.
- Направление на север в горизонтальном положении: `atan2(my, mx)`.
- На показания влияют моторы, магниты, металлические предметы и провода с током. Нужна калибровка: медленно вращать плату и найти минимумы и максимумы по осям, затем вычесть смещение.

Задание: зажигать светодиод, ближайший к направлению на север.

⚠️ Пример `Ardu8_16_IMU_Compass` на железе ещё не проверен.

Урок 15. Практикум «Уровень-пузырёк» или «Компас»

Примеры: Arduino IDE — `Ardu8_17_IMU_BubbleLevel`, `Ardu8_16_IMU_Compass`.

Ход урока: выбор задания — 5 мин, выполнение — 30 мин, демонстрация — 10 мин.

Конспект:

- «Уровень-пузырёк»: горящий светодиод смещается в сторону наклона, в горизонтальном положении RGB зелёный.
- «Компас»: светодиоды показывают направление на север, RGB — точность.
- Фильтр сглаживания: `x = x * 0.9 + новое * 0.1` убирает дрожание показаний.

Задание: добавить звуковой или световой сигнал при горизонтальном положении.

Урок 16. Зачёт по модулю 1

Ход урока: задание — 5 мин, выполнение — 35 мин, показ и вопросы — 5 мин.

Конспект (критерии):

- Программа работает по заданию, DIP-переключатель в правильном положении.
- Учащийся объясняет, как работают 74HC595, 74HC165 и I2C.
- Примеры заданий: «Кодовый замок», «Кубик» (случайное число по кнопке), «Индикатор наклона».

Модуль 2. Датчики Kirus (9 ч)

Урок 17. Модуль «Концевой выключатель», цифровые входы

Цель: читать цифровой датчик.

Подключение: модуль → D7 (XP21). Скетч `Ardu8_Kirus_uSW`.

Примеры: Arduino IDE — `Ardu8_18_Endstop`; AR BBL — `Ardu8_18_Endstop`.

Конспект:

- Концевой выключатель срабатывает, когда механизм доходит до края или робот касается препятствия.
- Цифровой вход понимает два уровня: HIGH (около 5 В) и LOW (около 0 В). Вход «в воздухе» даёт случайные значения, поэтому нужна подтяжка.
- В модуле есть подтяжка 10 кОм к 5 В: **не нажат — HIGH, нажат — LOW.** Зелёный светодиод модуля горит при нажатии.
- Разъёмы X2 и X4 одинаковые и совпадают с 3-пиновыми разъёмами контроллера.
- В скетче: LED1 и RGB показывают нажатие, счётчик нажатий с антидребезгом, BUT1 — сброс.

Задание: после 10 нажатий мигать красным.

Урок 18. Концевой выключатель как бампер

Цель: использовать датчик для обнаружения столкновений.

Примеры: Arduino IDE — `Ardu8_19_Bumper`; AR BBL — `Ardu8_19_Bumper`.

Конспект:

- Бампер — пластина перед роботом, которая нажимает концевой выключатель при столкновении.
- Считаем столкновения (события HIGH → LOW), а не время нажатия.
- Реакция на удар должна быть мгновенной: не используем длинные `delay()`, опрашиваем датчик в каждом проходе `loop()`.
- Можно запоминать время последнего удара и показывать «серию».

Задание: счётчик столкновений с выводом числа на LED1–LED4 в двоичном виде.

Урок 19. Модуль «Датчик линии»

Цель: понять принцип работы оптического датчика и настроить его.

Подключение: S1 (X4) → D3 (XP17), S2 (X2) → D4 (XP18). Скетч `Ardu8_Kirus_Line`.

Примеры: Arduino IDE — `Ardu8_20_LineSensor`; AR BBL — `Ardu8_20_LineSensor`.

Конспект:

- Два оптрона TCRT5000: ИК-светодиод и фототранзистор. Белое отражает ИК-свет хорошо, чёрное — плохо.
- Компаратор LM293 превращает сигнал в цифровой: **белое — LOW (горит светодиод канала), чёрное — HIGH.**
- Порог общий для двух каналов, задаётся подстроечником R7. Настраивать на том поле и при том освещении, где будет работать робот.
- Высота над полем — несколько миллиметров. Если датчик поднят высоко, он плохо различает цвета.
- Скетч показывает каналы на LED1/LED2 и цветом RGB.

Задание: найти положение R7, при котором датчик уверенно различает линию и фон.

Урок 20. Ситуации на поле

Цель: распознавать положение робота относительно линии.

Примеры: Arduino IDE — `Ardu8_21_LinePositions`; AR BBL — `Ardu8_21_LinePositions`.

Конспект:

- Четыре сочетания двух каналов: оба белые — «линии нет» или «линия между датчиками»; S1 чёрный — линия со стороны S1; S2 чёрный — со стороны S2; оба чёрные — перекрёсток или стоп-линия.
- Удобно собрать два бита в одно число: `state = s1 * 2 + s2` и использовать `switch`.
- Перекрёсток считаем по событию «оба стали чёрными», иначе один перекрёсток засчитается много раз.

Задание: вести датчик рукой по полю и считать перекрёстки.

Урок 21. Модуль «Ультразвуковой дальномер»

Цель: измерять расстояние ультразвуком.

Подключение: TRIG (X2) → D5 (XP19), ECHO (X4) → D6 (XP20). Скетч `Ardu8_Kirus_Ultrasonic`.

Примеры: Arduino IDE — `Ardu8_22_Ultrasonic`; AR BBL — `Ardu8_22_Ultrasonic`.

Конспект:

- Импульс 10 мкс на TRIG — дальномер излучает ультразвук 40 кГц. На ECHO появляется импульс длительностью, равной времени полёта звука туда и обратно.
- `pulseIn(6, HIGH, 30000)` — длительность импульса в микросекундах с ограничением ожидания.
- Звук проходит 1 см примерно за 29 мкс, туда и обратно — 58 мкс: **расстояние, см = мкс / 58**.
- Светодиоды модуля: ECHO мигает во время измерений, TRIG горит, когда программа держит линию в LOW.
- В скетче: шкала на LED1–LED4, зоны цвета RGB, график в плоттере.

Задание: сравнить показания с рулеткой на 10, 30, 50, 100 см.

Урок 22. Ошибки измерения

Цель: понять, откуда берутся ошибки дальномера, и научиться с ними бороться.

Примеры: Arduino IDE — `Ardu8_23_UltrasonicFilter`.

Конспект:

- Ультразвук плохо отражается от мягких предметов (ткань, поролон) и уходит в сторону от наклонных поверхностей. От узких предметов отражается слабо.
- Конус излучения широкий: дальномер «видит» не только то, что прямо впереди.
- Если эхо не вернулось, `pulseIn` возвращает 0 — это «нет данных», а не 0 см.
- Медианный фильтр из трёх измерений убирает одиночные выбросы лучше, чем среднее.
- Измерять чаще, чем раз в 30–60 мс, нельзя: отражения от прошлого импульса мешают новому.

Задание: составить таблицу «предмет — измерено — реально».

Урок 23. Все датчики вместе

Цель: подключить все модули одновременно.

Скетч: `Ardu8_Kirus_All`.

Примеры: Arduino IDE — `Ardu8_24_AllSensors`; AR BBL — `Ardu8_24_AllSensors`.

Конспект:

- Модули занимают D3–D7 и не мешают друг другу. D2 оставлен свободным для библиотеки моторов.
- Плоттер: подписи `Distance_cm`, `Line_S1`, `Line_S2`, `Endstop` — каждая становится отдельной линией.
- Ожидание ECHO ограничено по времени, чтобы остальные датчики опрашивались часто.

Задание: записать «показания» датчиков для трёх ситуаций и объяснить графики.

Урок 24. Проект «Парктроник»

Примеры: Arduino IDE — `Ardu8_25_Parktronic`; AR BBL — `Ardu8_25_Parktronic`.

Ход урока: постановка задачи — 5 мин, разработка — 30 мин, демонстрация — 10 мин.

Конспект:

- Чем ближе препятствие, тем чаще сигнал: больше 50 см — зелёный, 20–50 см — жёлтый, меньше 20 см — красный и частое мигание.
- Период мигания вычисляется из расстояния (например, `расстояние * 10` мс).
- Звук будет добавлен в модуле 3 (пищалка стоит на ESP32).

Задание: «режим тишины» по кнопке.

Урок 25. Проект на выбор

Ход урока: выбор и план — 10 мин, выполнение — 30 мин, показ — 5 мин.

Конспект (варианты):

- «Охранная сигнализация»: дальномер следит за дверным проёмом, концевой выключатель — за окном; включение и выключение кодом на кнопках.
- «Счётчик посетителей»: дальномер считает проходы.
- «Измеритель роста»: дальномер над головой, результат в мониторе порта.
- План: что делает устройство → какие датчики → схема состояний → код → проверка.

Модуль 3. ESP32 (9 ч)

Урок 26. Прошивка ESP32

Цель: научиться программировать ESP32.

Настройка: нижний USB-C, плата *ESP32 Wrover Module*.

Примеры: Arduino IDE — `Ardu8_26_ESP32_Leds`; AR BBL — `Ardu8_26_ESP32_Leds`.

Конспект:

- ESP32 работает от 3,3 В, имеет Wi-Fi и Bluetooth, гораздо быстрее Arduino-части.
- Загрузка через нижний USB-C. Если загрузка не начинается — держать BOOT при подключении; EN — перезапуск.
- Светодиоды подключены прямо к выводам: IO27, IO14, IO12, IO13 — зелёные LED1–LED4; IO15 — красный, IO2 — зелёный, IO18 — синий цвет RGB. **LOW — горит.**
- Датчики с выходом 5 В подключать к ESP32 напрямую нельзя.

Задание: бегущий огонь на светодиодах ESP32.

Урок 27. Кнопки и пищалка ESP32

Цель: работать с кнопками и звуком.

Примеры: Arduino IDE — `Ardu8_27_ESP32_Piano`, `Ardu8_28_ESP32_Melody`; AR BBL — `Ardu8_27_ESP32_Piano`.

Конспект:

- Кнопки: BUT1 — IO34, BUT2 — IO35, BUT3 — IO25, BUT4 — IO26. **Нажатие — HIGH.**
- На плате слева направо кнопки стоят в **обратном** порядке: BUT4, BUT3, BUT2, BUT1.
- IO34 и IO35 работают только как входы.
- Пищалка пассивная, на IO19: `tone(19, частота, длительность)`. Ноты: до — 262 Гц, ре — 294, ми — 330, фа — 349, соль — 392, ля — 440, си — 494.
- Мелодия — массивы нот и длительностей.

Задание: «пианино» и мелодия из 8 нот.

Урок 28. Wi-Fi: точка доступа и веб-страница ⚠️

Цель: управлять светодиодами с телефона.

Примеры: Arduino IDE — `Ardu8_29_ESP32_WiFi`; AR BBL — `Ardu8_29_ESP32_WiFi`.

Конспект:

- ESP32 может сам создать Wi-Fi-сеть (точку доступа): `WiFi.softAP("имя", "пароль")`. Адрес платы в этой сети — 192.168.4.1.
- Веб-сервер (`WebServer server(80)`) отвечает на запросы браузера: адрес `/` — страница, `/on` и `/off` — команды.
- Страница — HTML-текст с кнопками-ссылками.
- В `loop()` обязательно вызывать `server.handleClient()`.

Задание: кнопки на странице включают каждый светодиод ESP32.

⚠️ Пример Arduino IDE `Ardu8_29_ESP32_WiFi` на железе ещё не проверен; вариант для AR BBL проверен.

Урок 29. UART: передача Arduino → ESP32 ⚠️

Цель: передавать данные между двумя контроллерами платы.

Настройка: DIP2 = ON (прошивать Arduino при DIP2 = OFF). Скетч `Ardu8_Kirus_Sender`.

Примеры: Arduino IDE — `Ardu8_30_UART_Sender`, `Ardu8_31_UART_Receiver`; AR BBL — `Ardu8_30_UART_Sender`.

Конспект:

- UART передаёт байты по одному проводу в каждую сторону. DIP2 соединяет Arduino TX (D1) с ESP32 RX (IO22).
- Arduino отправляет строку `K,S1,S2,см,uSW` на скорости 115200 несколько раз в секунду.
- Буква `K` в начале — признак, что это наша строка. Конец строки — символ перевода строки.
- ESP32 принимает: `Serial2.begin(115200, SERIAL_8N1, 22, 23)`.

Задание: вывести принятые строки в монитор порта ESP32.

Урок 30. Датчики на веб-странице ⚠️

Цель: показать показания датчиков на телефоне.

Скетч: `Ardu8_esp_Kirus_Receiver`.

Примеры: Arduino IDE — `Ardu8_32_UART_WebStation`.

Конспект:

- ESP32 разбирает строку по запятым и сохраняет значения в переменных.
- Wi-Fi-точка «Ardu8-Kirus», страница http://192.168.4.1 показывает расстояние, линию и концевой выключатель.
- Страница обновляется автоматически (раз в секунду).
- Светодиоды и пищалка ESP32 реагируют на данные.

Задание: добавить на страницу цветовую индикацию расстояния.

⚠️ Скетч ESP32 не проверен на железе.

Урок 31. Мини-проект «Умная станция»

Примеры: Arduino IDE — `Ardu8_29_ESP32_WiFi`, `Ardu8_32_UART_WebStation`; AR BBL — `Ardu8_29_ESP32_WiFi`.

Ход урока: план — 10 мин, выполнение — 30 мин, показ — 5 мин.

Конспект:

- Станция собирает данные с датчиков и IMU, показывает их на телефоне и подаёт сигналы.
- Варианты: «Сторож» (сигнал на телефон при приближении), «Метеостанция-наклономер», «Счётчик проходов».
- Arduino-часть отвечает за датчики, ESP32 — за Wi-Fi и звук.

Урок 32. Зачёт за полугодие

Конспект (критерии):

- Правильные подключение и настройка DIP.
- Программа работает, учащийся объясняет код.
- Знание: шины SPI, I2C, UART; логика уровней датчиков; формула дальномера.

Уроки 33–34. Резерв

Отработка пропущенных тем, доработка проектов, пересдача зачёта.

II полугодие — моторы, сервоприводы, манипулятор (34 ч)

Модуль 4. Моторы и мобильный робот (16 ч)

Урок 35. Шина DXL, протокол Dynamixel 1.0

Цель: понять, как устроен обмен по шине.

Примеры: Arduino IDE — `Ardu8_33_DXL_Ping`.

Конспект:

- Моторами управляют два контроллера STM8, подключённые к шине DXL — одному проводу, по которому данные идут в обе стороны по очереди (полудуплекс).
- Каждое устройство имеет свой ID и отвечает только на «свои» пакеты.
- Пакет: `FF FF ID LEN INST PARAMS… CHK`. LEN = число параметров + 2.
- Команды: PING (1), READ (2), WRITE (3), SYNC_WRITE (0x83). Широковещательный ID — 0xFE.
- Контрольная сумма: `CHK = ~(ID + LEN + INST + PARAMS)`, младший байт. Если сумма не сошлась, модуль пакет не выполняет.
- Многобайтные числа передаются младшим байтом вперёд.

Задание: составить вручную пакет «записать 400 в регистр 28 модуля 71».

Урок 36. Поиск модулей

Цель: найти модули на шине и проверить их работу.

Настройка: DIP3 = ON, DIP4 = OFF. Скетч `Ardu8_esp_motors_test` (ESP32). Питание от аккумулятора.

Примеры: Arduino IDE — `Ardu8_33_DXL_Ping`.

Конспект:

- Модули моторов: **ID 71 и 72**, скорость 57600. ESP32: `Serial2.begin(57600, SERIAL_8N1, 33, 32)`, отладка — через `Serial`.
- Разъёмы моторов на правом краю, сверху вниз: 71/M1, 71/M2, 72/M1, 72/M2.
- Каналы видны по светодиодам у драйверов даже без моторов: при вращении одна пара светится ярко, другая тусклее, при остановке горят оба.
- ID модулей задан при заводской прошивке и по шине не меняется.

Задание: заполнить таблицу «разъём — ID — канал».

Урок 37. Мощность и направление

Цель: управлять моторами в режиме ШИМ.

Примеры: Arduino IDE — `Ardu8_34_DXL_Power`.

Конспект:

- Регистры: 28 — мощность M1, 30 — M2, от −1023 до 1023. Знак — направление, 0 — стоп. Запись включает режим ШИМ.
- ШИМ: питание мотора быстро включается и выключается. Чем больше доля «включено», тем быстрее вращение.
- При малой мощности мотор может не тронуться: не хватает силы на преодоление трения.
- Одна команда WRITE может записать сразу M1 и M2 (4 байта, начиная с регистра 28).

Задание: найти минимальную мощность, при которой мотор начинает вращаться.

Урок 38. Энкодеры и одометрия

Цель: измерять пройденный путь.

Примеры: Arduino IDE — `Ardu8_35_DXL_Encoder`.

Конспект:

- Энкодер считает импульсы при вращении вала. Регистры 32 (M1) и 36 (M2), число со знаком (int16).

- Тики на оборот колеса определяем опытом: 10 оборотов вручную, делим на 10.
- Путь = тики / (тики на оборот) × π × диаметр колеса.
- Одометрия — определение пройденного пути и положения робота по энкодерам.
- Счётчик ограничен диапазоном −32768…32767 и при переполнении «перескакивает». Для длинных поездок считаем разницу между соседними измерениями.
- Если энкодер считает в обратную сторону, его можно инвертировать регистром 6.

Задание: проехать 50 см и 1 м, измерить ошибку.

Урок 39. Поворот на угол по энкодерам

Цель: поворачивать робота на заданный угол.

Примеры: Arduino IDE — `Ardu8_36_DXL_TurnEncoder`.

Конспект:

- При танковом развороте колёса движутся по окружности диаметром, равным колее робота (расстоянию между колёсами).
- Путь колеса для поворота на угол α: `π × колея × α / 360`.
- Ошибки: проскальзывание колёс, инерция после выключения моторов. Перед целью надо снижать скорость.
- Поправочный коэффициент подбирается опытом.

Задание: «квадрат» 50×50 см и возвращение в исходную точку.

Урок 40. Режим скорости, PID-регулятор

Цель: держать заданную скорость с помощью регулятора.

Примеры: Arduino IDE — `Ardu8_37_DXL_SpeedPID`.

Конспект:

- Регистры 40 (M1) и 42 (M2) — заданная скорость в тиках за 100 мс. Запись включает режим скорости. Текущая скорость — регистры 34 и 38.
- Ошибка = заданная − текущая. P (регистр 47, по умолчанию 16) — реакция на ошибку сейчас, I (48, по умолчанию 32) — на накопленную, D (49, по умолчанию 0) — на изменение.
- Большой P — рывки и колебания, маленький — вялый разгон. Без I скорость не дотягивает до заданной.
- Регистр 24 — ограничение мощности, 45 — период регулятора (по умолчанию 50 мс). Регистры PID сбрасываются при выключении питания.

Задание: графики заданной и текущей скорости при трёх значениях P.

Урок 41. Моторы с Arduino через DxlMaster

Цель: управлять моторами из Arduino-части.

Настройка: прошивать при DIP4 = OFF; работа — DIP4 = ON, DIP1 = OFF, DIP3 = OFF. Скетч `Ardu8_dxlmaster_test`.

Примеры: Arduino IDE — `Ardu8_38_DxlMaster_Buttons`; AR BBL — `Ardu8_38_DxlMaster_Buttons`.

Конспект:

- Библиотека DxlMaster (Applied Robotics), протокол 1.0: `DynamixelDevice drv(71); drv.write(28, (int16_t)400); drv.read(32, enc);`.
- Шина висит на том же `Serial`, что и USB: `Serial.print` попадает на шину. Для отладки — светодиоды.
- Библиотека занимает **D2**, датчики к нему не подключаем.
- Теперь датчики (Arduino-часть) и моторы управляются одной программой.

Задание: управление шасси кнопками: вперёд, назад, влево, вправо.

Урок 42. Поворот по гироскопу

Цель: точный поворот с помощью IMU.

Примеры: Arduino IDE — `Ardu8_39_GyroTurn`.

Конспект:

- Гироскоп измеряет реальный поворот корпуса, поэтому проскальзывание колёс ему не мешает.
- Перед поворотом: робот неподвижен, измеряем смещение гироскопа. Во время поворота интегрируем угол и останавливаемся у цели.
- Скорость поворота снижаем пропорционально оставшемуся углу.
- Сравнение с уроком 39: какой способ точнее и почему.

Задание: развернуться на 90°, 180° и 360°, измерить ошибку.

Урок 43. Подъезд к стене по дальномеру

Цель: освоить П-регулятор.

Примеры: Arduino IDE — `Ardu8_40_WallStop`; AR BBL — `Ardu8_40_WallStop`.

Конспект:

- Задача: остановиться ровно в 20 см от стены.
- «Включить/выключить» (стоп при <20 см) — робот проскакивает цель по инерции.
- П-регулятор: `скорость = K × (расстояние − 20)`. Чем ближе к цели, тем медленнее едет робот.
- Скорость ограничивается сверху. Слишком большой K — робот раскачивается взад-вперёд у цели.

Задание: подобрать K и показать остановку с точностью ±1 см.

Урок 44. Езда вдоль стены

Цель: применить регулятор для движения на заданном расстоянии.

Примеры: Arduino IDE — `Ardu8_41_WallFollow`.

Конспект:

- Дальномер ставится сбоку (или под углом вперёд-вбок).
- Ошибка = расстояние − заданное. Поправка = K × ошибка: одно колесо ускоряется, другое замедляется.
- Скорость движения вперёд постоянна, регулятор управляет только разницей скоростей.
- Проблемы: углы стены, проёмы (резкий рост расстояния) — ограничиваем поправку.

Задание: проехать вдоль стены 2 м на расстоянии 15 см.

Урок 45. Робот по линии: сборка

Цель: подготовить шасси к езде по линии.

Примеры: Arduino IDE — `Ardu8_24_AllSensors`, `Ardu8_38_DxlMaster_Buttons`; AR BBL — `Ardu8_24_AllSensors`, `Ardu8_38_DxlMaster_Buttons`.

Конспект:

- Датчик линии — впереди по центру, низко над полем; дальномер — вперёд; концевой выключатель — бампер.
- Подключение: S1 → D3, S2 → D4, TRIG → D5, ECHO → D6, концевой выключатель → D7. D2 не занимать.
- Проверка: команда «вперёд» вращает оба колеса вперёд; определить, какой канал (S1 или S2) слева. Настроить R7 на поле.

Задание: записать знаки моторов и сторону S1 своего робота.

Урок 46. Робот по линии: алгоритм на двух датчиках ⚠️

Цель: запрограммировать движение по линии.

Скетч: `Ardu8_Kirus_LineFollower` (DIP4 = ON).

Примеры: Arduino IDE — `Ardu8_42_LineFollower`; AR BBL — `Ardu8_42_LineFollower`.

Конспект:

- Датчики по краям линии. Оба белые — прямо. Левый чёрный — влево. Правый чёрный — вправо. Оба чёрные — перекрёсток.
- Чем выше скорость, тем чаще вылеты на поворотах. Плавный поворот — замедляем внутреннее колесо, резкий — вращаем его назад.
- Запоминаем, в какую сторону ушла линия: если оба датчика потеряли её, поворачиваем в ту же сторону.

Задание: пройти трассу без вылетов.

⚠️ Скетч — черновик.

Урок 47. Перекрёстки, препятствие, бампер

Цель: усложнить поведение робота.

Примеры: Arduino IDE — `Ardu8_43_LineObstacle`, `Ardu8_44_LineCrossings`; AR BBL — `Ardu8_43_LineObstacle`.

Конспект:

- Перекрёсток (оба чёрные): считаем и по номеру решаем — прямо, налево, направо.
- Проезд перекрёстка: короткий проезд вперёд по энкодерам, затем поворот до появления линии.
- Препятствие ближе 15 см — стоп и ожидание. Бампер — стоп и отъезд назад.

Задание: маршрут «на 2-м перекрёстке налево, на 3-м — стоп».

Урок 48. Тренировочные заезды

Примеры: Arduino IDE — `Ardu8_42_LineFollower`; AR BBL — `Ardu8_42_LineFollower`.

Ход урока: настройка — 15 мин, заезды на время — 25 мин, разбор — 5 мин.

Конспект:

- Подбираем максимальную скорость, при которой робот не вылетает.
- Ведём протокол: скорость, время, число вылетов.
- Меняем по одному параметру за раз, иначе непонятно, что помогло.

Урок 49. Управление роботом с телефона ⚠️

Цель: объединить Wi-Fi и моторы.

Настройка: DIP3 = ON, DIP4 = OFF (моторами управляет ESP32).

Примеры: Arduino IDE — `Ardu8_45_PhoneRobot`.

Конспект:

- ESP32 создаёт точку доступа и веб-страницу с кнопками «вперёд», «назад», «влево», «вправо», «стоп».
- Нажатие кнопки на странице отправляет запрос, ESP32 записывает скорости в регистры 40/42 модулей.
- Безопасность: если команды не приходят больше 0,5 с — робот останавливается.

Задание: проехать «змейку» между кеглями.

⚠️ Пример `Ardu8_45_PhoneRobot` на железе ещё не проверен.

Урок 50. Резерв / практикум

Доработка робота, повторные заезды, отработка пропущенных тем.

Модуль 5. Сервоприводы Feetech ST3215 (6 ч)

Урок 51. Устройство и питание привода

Цель: подключить сервопривод, найти его и назначить ID.

Настройка: DIP3 = ON, DIP4 = OFF, привод в XP2 или XP3. Скетчи `Ardu8_ST3215_Scan`, `Ardu8_ST3215_SetID`.

Примеры: примеры библиотеки FeetechSTS — `Ardu8_ST3215_Scan`, `Ardu8_ST3215_SetID`.

Конспект:

- Сервопривод — мотор, редуктор, датчик угла и контроллер в одном корпусе. Он сам удерживает заданный угол.
- ST3215 подключается к шине DXL, но работает на **1 000 000 бит/с** и использует свою таблицу регистров. Модули моторов (57600) ему не мешают.
- **Напряжение аккумулятора должно подходить приводу** (бывают на 7,4 и на 12 В): питание идёт с входа без стабилизации.
- Заводской ID — 1. Смена ID — при одном приводе на шине: `servo.changeId(новый)`. **ID 71 и 72 не использовать.**
- Библиотека FeetechSTS ставится через «Добавить .ZIP библиотеку».

Задание: назначить приводу ID, выданный преподавателем, подписать привод.

Урок 52. Веб-настройщик

Цель: настраивать привод через веб-страницу.

Скетч: `Ardu8_Feetech_Bridge` (ESP32), страница `feetech_tool.html`.

Примеры: мост `Ardu8_Feetech_Bridge` и страница `feetech_tool.html`.

Конспект:

- Страница работает в Chrome / Edge / Яндекс Браузере (нужен Web Serial) без интернета. Режим «Ardu8», порт нижнего USB-C. Монитор порта Arduino IDE закрыть.
- Индикация моста: LED1 ESP32 — мост работает, LED2 — обмен, RGB зелёный/красный — привод ответил / не ответил.
- Раздел «Один привод»: поиск, телеметрия, круг положения с перетаскиванием цели, удержание и отпускание, режимы, смена ID и скорости, журнал пакетов.
- Параметры EEPROM сохраняются при выключении. Для записи их нужно разблокировать. Менять только то, что понимаешь.
- Режим «Демо» позволяет потренироваться без привода.

Задание: записать в тетрадь напряжение, температуру и положение привода в покое и под нагрузкой рукой.

Урок 53. Режим позиции

Цель: поворачивать вал на заданный угол.

Скетч: `Ardu8_ST3215_Position`.

Примеры: Arduino IDE — `Ardu8_46_ST3215_Metronome`; примеры библиотеки FeetechSTS — `Ardu8_ST3215_Position`.

Конспект:

- Оборот — 4096 шагов, 2048 — середина (180°). 1° ≈ 11,4 шага.
- `bus.begin(Serial2, 1000000, 33, 32)`; `FeetechServo servo(bus, 1)`; `servo.setMode(STS_MODE_POSITION)`.
- При старте — **`servo.hold()`**: текущий угол становится целью, вал не дёргается.
- `servo.moveToDeg(угол, скорость, ускорение)`, например `moveToDeg(270, 2000, 50)`.
- Кнопки ESP32 в примере: BUT1 — 90°, BUT2 — 180°, BUT3 — 270°, BUT4 — отпустить вал. RGB зелёный во время движения.

Задание: «метроном» с регулировкой скорости кнопками.

Урок 54. Обратная связь

Цель: использовать данные привода в программе.

Примеры: Arduino IDE — `Ardu8_47_ST3215_Feedback`, `Ardu8_48_ST3215_Teach`.

Конспект:

- Привод сообщает положение, скорость, нагрузку, напряжение, температуру и ток: `readPosition()`, `readLoad()`, `readTemperature()`, `readCurrent()` или всё сразу `readState(st)`.
- «Захват с ограничением усилия»: закрываем схват маленькими шагами; когда нагрузка превышает порог — останавливаемся. Предмет держится, а мотор не перегревается.
- Температура выше нормы — сигнал снизить нагрузку или дать приводу остыть.
- «Обучение показом»: отпустить вал, повернуть рукой, прочитать положение — и потом повторить движение.

Задание: записать рукой 5 положений и воспроизвести их.

Урок 55. Режим колеса

Цель: использовать привод как мотор.

Скетч: `Ardu8_ST3215_Wheel`.

Примеры: Arduino IDE — `Ardu8_49_ST3215_Conveyor`; примеры библиотеки FeetechSTS — `Ardu8_ST3215_Wheel`.

Конспект:

- `servo.setMode(STS_MODE_WHEEL)` и `servo.setSpeed(шагов/с)` — непрерывное вращение со знаком направления.
- Режим сохраняется в приводе: перед работой в режиме позиции его нужно вернуть.
- Применение: колёса, конвейер, лебёдка.

Задание: «конвейер» — вращение с остановкой по концевому выключателю.

Урок 56. Несколько приводов ⚠️

Цель: управлять несколькими приводами одновременно.

Скетч: `Ardu8_ST3215_Sync`.

Примеры: Arduino IDE — `Ardu8_50_ST3215_Mirror`; примеры библиотеки FeetechSTS — `Ardu8_ST3215_Sync`.

Конспект:

- Команды по очереди → приводы стартуют с задержкой и приходят в цель в разное время.
- SYNC WRITE — один пакет с данными для всех приводов: `bus.syncMoveTo(...)`. Чтение положений — `bus.syncReadPositions(...)`.
- Чтобы приводы пришли в цель одновременно, скорость каждого рассчитывается пропорционально его пути.

Задание: два привода качаются «в зеркало», затем «волной».

⚠️ Скетчи с SYNC-командами на железе не проверены (в веб-настройщике SYNC WRITE проверен).

Модуль 6. Манипулятор (12 ч)

Урок 57. Устройство манипулятора

Цель: познакомиться с устройством манипулятора и понятием степеней свободы.

Конспект:

- Манипулятор — «рука» робота. Каждый сустав даёт одну степень свободы.
- Суставы снизу вверх: 1 — поворот основания, 2 — плечо, 3 — локоть, 4 — поворот предплечья, 5 — наклон кисти, 6 — поворот кисти, 7 — схват. ID привода равен номеру сустава.
- Рабочая зона — все точки, до которых может дотянуться схват.
- «Поза модели» (все суставы 0°): плечо вертикально, предплечье горизонтально вперёд, кисть вниз, схват раскрыт.
- Раздел «Манипулятор» веб-настройщика показывает 3D-модель и позволяет двигать суставы ползунками (в «Демо» — без железа).

Задание: в «Демо» подобрать позу, в которой схват касается стола перед основанием.

Урок 58. Подготовка приводов

Цель: подготовить 7 приводов к сборке.

Конспект:

- Новые приводы имеют ID 1 — подключаем строго по одному.
- Помощник «Подготовка приводов»: найти привод → записать ID и скорость 1 Мбит/с → повернуть вал в 2048 → надеть фланец.
- Фланец: 4 отверстия крестом «+» вдоль и поперёк корпуса, центральный винт M3, риска маркером на фланце и корпусе, подпись «ID N». После «Фланец надет» момент снимается.
- Прогресс сохраняется в браузере.

Задание: подготовить все приводы группы.

Урок 59. Сборка: основание, плечо, локоть

Конспект:

- Собираем в «позе модели», риски на фланцах и корпусах совпадают.
- Шлицы фланца дают шаг 14,4°: промах на один зубец смещает сустав.
- Холостые фланцы (приводы 2–5) вращаются свободно, их ставят при сборке звена.
- Винты затягивать без перекоса, кабели укладывать с запасом на движение.

Задание: проверить вручную свободное движение собранных суставов.

Урок 60. Сборка: кисть, схват

Конспект:

- Кисть: поворот предплечья (4), наклон (5), поворот кисти (6).
- Схват (7) — шестерни и параллелограмм. **Шестерни ставить при полностью раскрытых губках.**
- Финальная проверка: все кабели подключены, ничего не натягивается во всём диапазоне движения.

Задание: сфотографировать манипулятор в «позе модели» для отчёта.

Урок 61. Калибровка суставов

Конспект:

- «Ноль = сейчас» (или «Принять текущую позу за позу модели») записывает текущее положение в привод как середину шкалы: ноль сустава становится 2048, диапазон остаётся по центру шкалы.
- Инверсия — если сустав вращается «не в ту сторону» по сравнению с 3D-моделью.
- Пределы — чтобы манипулятор не ударялся о себя и о стол.
- Настройки хранятся в браузере — обязательно сохранить их в JSON-файл.

Задание: откалибровать все суставы и сверить движение с 3D-моделью.

Урок 62. Позы и сценарии движений

Конспект:

- Поза — набор углов всех суставов. Её можно задать ползунками или запомнить из реального положения.
- Все суставы приходят в позу одновременно за «время перехода».
- Сценарий — последовательность поз, которую можно переставлять и проигрывать по кругу.
- Слишком короткое время перехода даёт рывки и перегрузку приводов.

Задание: сценарий «помахать рукой» и «поклон».

Урок 63. Основы кинематики

Цель: вычислить положение схвата по углам суставов.

Примеры: Arduino IDE — `Ardu8_51_Kinematics`.

Конспект:

- Прямая задача кинематики: по углам суставов найти, где находится схват.
- Плоская модель «плечо + локоть»: длины звеньев L1 (от оси плеча до оси локтя) и L2 (от оси локтя до кисти). По модели L1 ≈ 120 мм; L2 измеряем линейкой на своём манипуляторе.
- Если углы отсчитываются от горизонтали: `x = L1·cos(θ1) + L2·cos(θ1 + θ2)`, `y = L1·sin(θ1) + L2·sin(θ1 + θ2)`.
- Обратная задача — по нужной точке найти углы — сложнее и имеет два решения («локоть вверх» и «локоть вниз»).

Задание: рассчитать положение кисти для трёх поз и проверить линейкой.

Урок 64. Итоговый проект: тема и план

Конспект:

- Варианты: «Перенеси кубик из зоны A в зону B», «Сортировщик по цвету» (с ручной подсказкой), «Робот по линии + манипулятор», «Рисующая рука».
- План: цель → оборудование → алгоритм (схема) → роли в команде → критерии успеха → график работ на уроки 65–66.

Урок 65. Итоговый проект: разработка

Конспект:

- Делим задачу на части и делаем по одной: позы, переходы, взаимодействие с датчиками.
- Для переноса кубика: «домой» → над кубиком → опуститься → сжать схват (с ограничением усилия) → подняться → перенести → опустить → разжать → «домой».

Урок 66. Итоговый проект: отладка

Конспект:

- Отлаживаем по частям, затем весь цикл. Записываем, что исправили.
- Типичные проблемы: кубик выскальзывает, сустав перегревается, манипулятор задевает стол, робот съезжает с линии.
- Готовим презентацию: цель, устройство, алгоритм, трудности, результат.

Урок 67. Соревнование / защита проектов

Конспект (регламент):

- Заезды по линии: время прохождения трассы, штраф за вылет.
- Манипулятор: число кубиков, перенесённых за 2 минуты.

- Защита (3–5 минут): цель, устройство, алгоритм, трудности, демонстрация, ответы на вопросы.

Урок 68. Подведение итогов

Обсуждение результатов года, награждение, идеи для дальнейших проектов.

Примечания

⚠️ — уроки опираются на скетчи, которые ещё не проверены на железе: магнитометр (14), Wi-Fi-страница (28), передача данных в ESP32 (29–30), LineFollower (46), управление с телефона (49), SYNC (56). Примеры для уроков 14, 28, 49 и для уроков с IMU (12–15, 42) подготовлены в папке `lessons`. Все новые примеры Arduino IDE компилируются, но на железе ещё не проверены. Примеры AR BBL, совпадающие с набором `ar_bbl_examples`, проверены на контроллере 30.10.2026.